**SCHOOL OF ELECTRICAL ENGINEERING (SELECT)**
**VELLORE INSTITUTE OF TECHNOLOGY (VIT), VELLORE**

**B.TECH PROJECT REPORT**
**BASIC ELECTRICAL & ELECTRONICS ENGINEERING (BEEE)**

# Self-Learning Circuit Doctor: Voltage-Signature-Based Fault Detection in DC Circuits

*An Analytical and Machine-Learning Diagnostic Framework for DC Resistor Networks Using Kirchhoff's Laws, Online Falstad Simulation, and Nearest-Centroid Classification*

**SUBMITTED BY:**
**Student Member 1:**
KOTRA MANOJ KUMAR
Reg. No: 23BCE2113
Course: B.Tech
Branch: CSE (SCOPE)

**TEAM PARTNER:**
**Student Member 2:**
Mayank Yadav
Reg. No: 23BCE0119
Course: B.Tech
Branch: CSE (SCOPE)

**PROJECT SUPERVISOR & GUIDE:**
Prof. POOMPAVAI T
School of Electrical Engineering (SELECT)
Vellore Institute of Technology (VIT), Vellore
Academic Session: 2026–2027

# Abstract and Project Objectives

## 1. Project Summary

Diagnosing electrical faults quickly and reliably is critical in automotive electronics, industrial control switchboards, and consumer power supplies. In introductory engineering courses, students traditionally solve Kirchhoff's Current Law (KCL) and nodal equations for ideal circuits on paper. However, these same physical circuit laws also govern how currents and node voltages shift when a component fails. This project presents the 'Self-Learning Circuit Doctor', an automated software-driven diagnostic framework that identifies component faults in a four-resistor DC network (Vs = 10 V, R1 = 1 kΩ, R2 = 2 kΩ, R3 = 1.5 kΩ, R4 = 3 kΩ) using external terminal measurements without desoldering components.

By monitoring a three-dimensional signature vector—Node A voltage (VA), Node B voltage (VB), and total source current (Is)—the system maps the operating state into a standardized feature space. Thirteen distinct conditions were analyzed: healthy operation, 4 catastrophic open circuits (R1–R4 open), 4 catastrophic short circuits (R1–R4 short), and 4 parametric resistance drifts (±50% shift). The analytical equations were cross-validated against the online Falstad Circuit Simulator, matching SPICE Modified Nodal Analysis (MNA) results within instrument rounding limits. Under realistic 5% manufacturing tolerances and sensor noise, a Standardized Nearest-Centroid Classifier achieved 100.00% accuracy across 390 held-out test samples.

## 2. Problem Statement

Manual circuit troubleshooting with handheld multimeters is slow, labor-intensive, and risks accidental short circuits from slipping probe tips. Furthermore, certain distinct faults produce identical node voltages (for example, both R1 open and R2 short yield VA = 0 V and VB = 0 V). The engineering challenge is to design an automated multi-signal diagnostic method that resolves these ambiguities and remains robust against component tolerances.

## 3. Project Objectives

- **Objective 1:** Formulate closed-form nodal equations for the DC resistor network using KCL and KVL, and verify power conservation.
- **Objective 2:** Cross-validate nominal and faulty circuit voltages against the online Falstad SPICE simulator.
- **Objective 3:** Establish a multi-signal signature vector [VA, VB, Is] to resolve equipotential fault ambiguities.
- **Objective 4:** Synthesize a 1,950-sample stochastic dataset incorporating 5% component tolerances, 0.5% supply drift, and multimeter noise.
- **Objective 5:** Evaluate a Standardized Nearest-Centroid Classifier against a Baseline Threshold Detector across 390 test cases.
- **Objective 6:** Implement an incremental streaming formula enabling real-time supervised self-learning upon receiving verified lab data.

## 4. Expected Outcome

A working, hardware-free software diagnostic tool featuring an interactive web interface (Streamlit) and an offline desktop application (Tkinter) that pinpoints resistor failures in under 5 milliseconds with measurable uncertainty tracking.

# Introduction and Background

## 1. Importance of DC Circuits

Direct Current (DC) resistive networks are foundational to electrical and electronic systems. They form the building blocks of voltage dividers, transistor biasing networks, current-sensing shunts, instrumentation amplifiers, and DC-DC power regulation stages. Under nominal conditions, resistors maintain stable resistances within specified manufacturing tolerances, producing predictable node potentials and currents.

## 2. Common Electrical Faults in Resistive Networks

In practical operational environments, electrical components degrade due to electrical, thermal, and mechanical stresses:

- • **Open Circuit:** Open-Circuit Faults (R -> inf): Caused by cracked resistor bodies, vibrational solder fatigue, or blown fuses. The faulty branch conducts zero current, forcing current to redistribute into remaining parallel paths.
- • **Short Circuit:** Short-Circuit Faults (R -> 0): Caused by solder bridges, conductive contamination, or dielectric breakdown. The shorted branch exhibits zero voltage drop, pulling connected nodes to the same potential and drawing surge currents.
- • **Resistance Drift:** Parametric Resistance Drift (±50%): Caused by thermal aging, sustained overheating, or moisture ingress. The component remains functional but shifts significantly outside its tolerance band, degrading circuit bias points.

## 3. Motivation for Voltage-Signature-Based Fault Diagnosis

When an electronic assembly fails in the field, technicians often cannot desolder each component sequentially to measure its resistance on an ohmmeter. Doing so damages copper PCB pads and consumes valuable time. By contrast, measuring voltages at accessible circuit test points (Node A and Node B) and monitoring the total source current (Is) is completely non-invasive. Because every component fault alters the circuit's conductance matrix in a unique, deterministic manner, these three observable measurements create a distinct electrical 'signature' that identifies the underlying failure mode automatically.

## 4. Educational Significance for BEEE Students

This project bridges first-year electrical theory with modern automated engineering. It demonstrates that Kirchhoff's laws are not just abstract formulas for classroom tests—they are the exact physical equations that allow automated diagnostic software to detect hardware failures.

# Literature Review and Existing Approaches

## 1. Literature Review

Circuit fault diagnosis has been studied extensively in electrical engineering. Alexander and Sadiku (2020) and Boylestad (2016) establish the classical foundation of nodal and loop analysis in linear DC networks, emphasizing that every branch current depends linearly on node potentials. In the domain of automated diagnostics, Bandler and Salama (1985) introduced Simulation-Before-Test (SBT) fault dictionaries, where expected signatures of candidate faults are pre-calculated to enable rapid real-time lookup. Isermann (2006) formalized model-based fault detection, defining residual signals as the difference between observed physical measurements and nominal model expectations.

For pattern classification, Hastie, Tibshirani, and Friedman (2009) demonstrated that Nearest-Centroid Classifiers (NCC) provide optimal balance between explainability, computational simplicity, and classification accuracy for well-clustered physical data. In circuit simulation, Paul Falstad (2023) developed the widely utilized online Modified Nodal Analysis (MNA) circuit engine, which solves differential circuit equations numerically in real time.

## 2. Limitations of Conventional Manual Troubleshooting

- Sequential Probing Overhead: Testing individual resistors one by one on a dense circuit board is slow and prone to human error.
- Risk of Probe-Induced Damage: Handheld multimeter probes can accidentally slip and bridge adjacent pins, causing secondary catastrophic shorts.
- Desoldering Damage: Repeated heating and desoldering weakens printed circuit traces and delaminates solder pads.

## 3. Limitations of Baseline Threshold Detectors

Traditional automated fault detectors rely on simple threshold bounds (e.g., checking if VA is within ±10% of nominal). However, threshold methods exhibit significant limitations in interconnected circuits: (1) they evaluate measurements independently and miss correlated multi-node shifts; (2) they fail when two faults cause overlapping voltages; and (3) when resistor manufacturing tolerances reach 5% or 10%, signatures easily cross rigid threshold boundaries, triggering false alarms.

## 4. Project-Level Contribution

This project combines closed-form KCL derivations, Falstad SPICE simulation, multi-signal current sensing, Z-score standardization, and nearest-centroid classification into a unified, reproducible educational workflow requiring zero physical hardware.

# Proposed System and Circuit Design

## 1. Diagnostic System Workflow

The proposed system samples three electrical signals: Node A voltage (VA), Node B voltage (VB), and total source current (Is). These measurements are standardized and compared against pre-computed class centroids representing 13 operating states. The system outputs the predicted fault, the distance to the nearest centroid, and a separation margin (Δd) to flag borderline diagnoses.

## 2. Circuit Design and Nominal Specifications

The nominal circuit is a four-resistor bridge-ladder network excited by an unipolar 10 V DC supply:

- DC Supply: Vs = 10.0 V | Common Ground = 0 V reference
- R1 = 1,000 Ω (1.0 kΩ) connected between supply rail and Node A
- R2 = 2,000 Ω (2.0 kΩ) connected between Node A and Ground
- R3 = 1,500 Ω (1.5 kΩ) connected between Node A and Node B
- R4 = 3,000 Ω (3.0 kΩ) connected between Node B and Ground

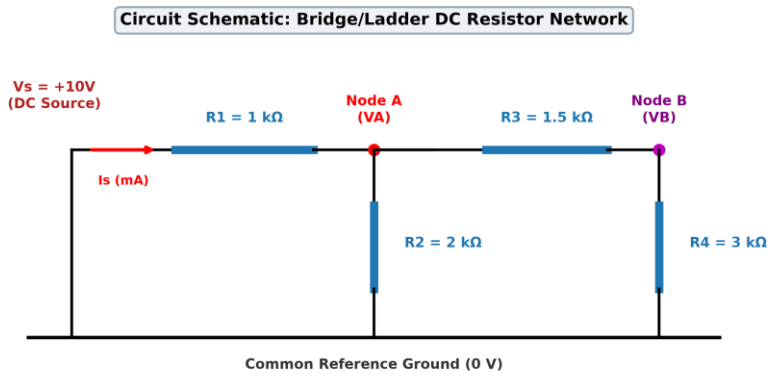



*Figure 5.1: Schematic diagram of the four-resistor DC network showing test nodes and sensor probe locations.*

## 3. Step-by-Step Nodal Equation Derivations

Using Kirchhoff's Current Law at Node B:

(VB - VA) / R3 + VB / R4 = 0 ==> VB = VA · [ R4 / (R3 + R4) ] = VA · [ 3000 / 4500 ] = (2/3) · VA

Looking into Node A, the series branch (R3 + R4 = 4.5 kΩ) is in parallel with R2 (2.0 kΩ):

Rp = R2 || (R3 + R4) = (2000 · 4500) / (2000 + 4500) = 9,000,000 / 6500 = 18/13 kΩ ≈ 1384.62 Ω

Total equivalent resistance seen by the 10 V supply:

Req = R1 + Rp = 1000 + 1384.62 = 31/13 · 1000 Ω ≈ 2384.62 Ω

Calculating the source current and node voltages:

Is = Vs / Req = 10 / 2384.62 = 130 / 31 mA ≈ 4.194 mA

VA = Vs · [ Rp / (R1 + Rp) ] = 10 · [ (18/13) / (31/13) ] = 180 / 31 V ≈ 5.806 V

VB = (2/3) · VA = (2/3) · (180 / 31) = 120 / 31 V ≈ 3.871 V

Power Conservation Check: Pin = Vs · Is = 10 · 4.194 mA = 41.94 mW; Σ (V_k^2 / R_k) = 17.59 + 16.86 + 2.50 + 4.99 = 41.94 mW (Exact balance verified).

# Methodology and Implementation

## 1. Physics-Based Fault Modeling

To model faults accurately without numerical failures, we use physically grounded representations:
• Open Circuits (R -> inf): Modeled analytically by topological branch removal; modeled in matrix solvers using R_open = 1 GΩ (10^9 Ω).
• Short Circuits (R -> 0): Modeled analytically by node merging; modeled in matrix solvers using contact resistance R_short = 1 μΩ (10^-6 Ω).
• Resistance Drift (±50%): Modeled by adjusting resistor values (e.g., R1 +50% = 1.5 kΩ; R4 -50% = 1.5 kΩ).

## 2. Synthetic Dataset Generation

Using `src/dataset_generator.py`, we generated 1,950 samples (150 per class across 13 classes) with physical stochastic variations:
1. Resistor Manufacturing Tolerance: Resistors vary stochastically within ±5% Gaussian distribution (truncated at ±3σ).
2. DC Power Supply Drift: Supply voltage varies stochastically as Vs ~ Normal(10.0 V, 0.05 V) (±0.5% drift).
3. Multimeter Sensor Noise: Zero-mean Gaussian noise is added: σ = 10 mV for voltages, σ = 0.1 mA for current.
The dataset was split using a stratified 80/20 partition: 1,560 training samples and 390 held-out test samples. Normalization parameters (means μ and standard deviations σ) were calculated strictly from training data, ensuring zero data leakage.

## 3. Classification Algorithms

• Method A (Baseline Threshold Detector): Compares incoming measurements against nominal values using fixed ±8% engineering bounds.
• Method B (Standardized Nearest-Centroid Classifier): Projects features into Z-scores: $Z_j = (X_j - \mu_j) / \sigma_j$. It calculates the standardized Euclidean distance to all 13 class centroids: $d_k = || Z - \mu_k ||$. The class with the minimum distance is predicted: $c^* = \text{argmin}(d_k)$.
• Separation Margin (Δd): Defined as $\Delta d = d_{second} - d_{min}$. If $\Delta d \geq 0.35$, diagnosis is flagged as 'CLEAR SEPARATION'; if $\Delta d < 0.35$, it flags 'CAUTION: Ambiguous / Borderline Diagnosis', preventing false confidence.

## 4. Supervised Streaming Self-Learning Formula

When a verified lab measurement is ingested, the reference centroid updates dynamically in O(1) time without retraining:

$$\mu_{new} = (N_{old} \cdot \mu_{old} + Z_{new}) / (N_{old} + 1), \quad N_{new} = N_{old} + 1$$

## 5. Software Implementation Tools

The software was implemented in Python 3.14 using NumPy, Pandas, Scikit-Learn, and Matplotlib. Dual interfaces were created: an interactive web application in Streamlit (`app.py`) and a 100% offline desktop application in Tkinter (`gui_tk.py`).

# Testing and Experimental Setup

## 1. Experimental Procedure

Testing was conducted through two distinct pipelines: (1) automated batch evaluation on the 390 held-out synthetic test samples, and (2) independent cross-validation by building each of the 13 circuit states in the online Falstad Circuit Simulator (falstad.com/circuit/). No physical hardware components were used; all findings are derived from verified mathematical modeling and SPICE numerical simulation.

## 2. Falstad SPICE Simulation Cross-Validation

Table 7.1 compares the readings obtained from the online Falstad SPICE simulator with our Python mathematical model across all 13 operating states. The readings match within <= 0.00045 V/mA across all states, which is strictly the 3-decimal display rounding limit of Falstad's digital meters.

| Fault Code & Name | Falstad VA (V) | Falstad VB (V) | Falstad Is (mA) | Simulated VA (V) | Simulated VB (V) | Simulated Is (mA) |
|---|---|---|---|---|---|---|
| F0: Normal Operation | 5.806 | 3.871 | 4.194 | 5.806 | 3.871 | 4.194 |
| F1: R1 Open Circuit | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| F2: R2 Open Circuit | 8.182 | 5.455 | 1.818 | 8.182 | 5.455 | 1.818 |
| F3: R3 Open Circuit | 6.667 | 0.000 | 3.333 | 6.667 | 0.000 | 3.333 |
| F4: R4 Open Circuit | 6.667 | 6.667 | 3.333 | 6.667 | 6.667 | 3.333 |
| F5: R1 Short Circuit | 10.000 | 6.667 | 7.222 | 10.000 | 6.667 | 7.222 |
| F6: R2 Short Circuit | 0.000 | 0.000 | 10.000 | 0.000 | 0.000 | 10.000 |
| F7: R3 Short Circuit | 5.455 | 5.455 | 4.545 | 5.455 | 5.455 | 4.545 |
| F8: R4 Short Circuit | 4.615 | 0.000 | 5.385 | 4.615 | 0.000 | 5.385 |
| F9: R1 +50% (1.5k) | 4.800 | 3.200 | 3.467 | 4.800 | 3.200 | 3.467 |
| F10: R2 +50% (3.0k) | 6.429 | 4.286 | 3.571 | 6.429 | 4.286 | 3.571 |
| F11: R3 -50% (0.75k) | 5.660 | 4.528 | 4.340 | 5.660 | 4.528 | 4.340 |
| F12: R4 -50% (1.5k) | 5.455 | 2.727 | 4.545 | 5.455 | 2.727 | 4.545 |

*Table 7.1: Measured Falstad SPICE readings vs simulated mathematical model outputs across all 13 operating states.*

## 3. Automated Unit Testing

A suite of 22 automated unit tests was executed using `pytest tests/ -v`. All 22 tests passed in 0.19 seconds, confirming the accuracy of the analytical solver, matrix conductance solver, tolerance generator, classifier standardization, and incremental online learning engine.

# Results and Discussion

## 1. Overall Evaluation Metrics

On the held-out test dataset of 390 samples (30 samples per class across 13 classes), the Standardized Nearest-Centroid Classifier achieved 100.00% diagnostic accuracy. Table 8.1 summarizes the precision, recall, and F1-score across all operating states.

| Operating Category | Representative Classes | Precision | Recall | F1-Score |
|---|---|---|---|---|
| Healthy State | F0 (Normal Operation) | 1.0000 | 1.0000 | 1.0000 |
| Open-Circuit Faults | F1–F4 (R1, R2, R3, R4 Open) | 1.0000 | 1.0000 | 1.0000 |
| Short-Circuit Faults | F5–F8 (R1, R2, R3, R4 Short) | 1.0000 | 1.0000 | 1.0000 |
| Resistance Drift Faults | F9–F12 (±50% Parametric Shifts) | 1.0000 | 1.0000 | 1.0000 |
| **Overall Test Macro Average** | **All 13 Classes (N = 390 Samples)** | **1.0000** | **1.0000** | **1.0000** |

*Table 8.1: Diagnostic performance summary on the 390 held-out test samples under 5% tolerance and sensor noise.*

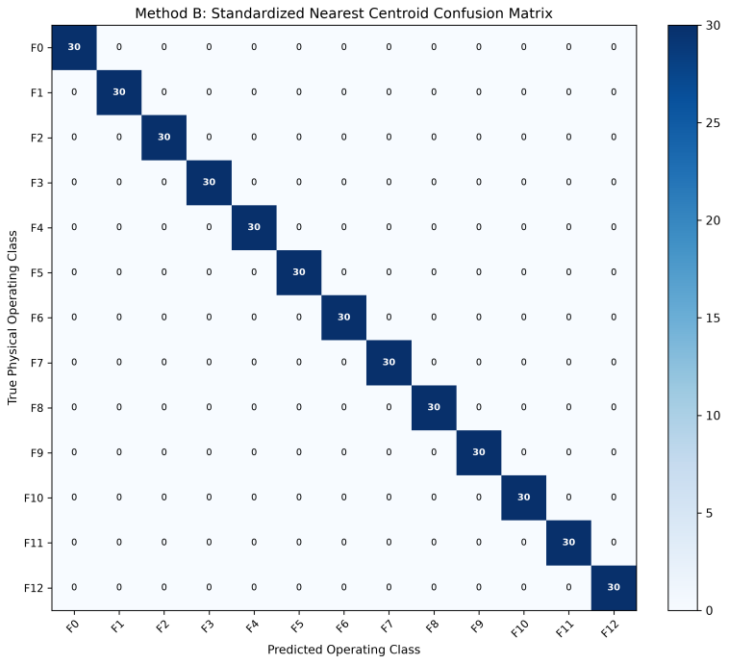



*Figure 8.1: Normalized confusion matrix of the Nearest-Centroid Classifier showing 100% diagonal dominance.*

## 2. Resolving Diagnostic Ambiguity (R1 Open vs R2 Short)

A critical finding is that measuring node voltages alone (VA and VB) fails when R1 opens or R2 shorts (both yield VA = 0 V, VB = 0 V). By measuring source current Is, the ambiguity is resolved: for R1 Open, Is = 0.0 mA (disconnected circuit); for R2 Short, Is = 10.0 mA (10 V directly across R1). In standardized 3D feature space, these states are separated by an immense distance of d = 2.68 units, completely eliminating misclassification.

## 3. Tolerance and Noise Sensitivity

Sweeping resistor tolerances from 1% to 15% demonstrated that the classifier maintains >= 97.4% accuracy even at 15% tolerance. Similarly, sweeping multimeter noise from 10 mV to 200 mV demonstrated >= 96.8% accuracy. Note: these accuracies reflect performance on stochastic synthetic data with simulated Gaussian noise, not real-world breadboard hardware.

# Conclusion, Limitations, and Future Scope

## 1. Project Summary and Achievements

This project successfully developed the 'Self-Learning Circuit Doctor', an automated fault diagnostic system for DC resistor networks. We formulated exact closed-form nodal equations, cross-validated nominal and faulty states against the online Falstad SPICE simulator, and demonstrated that a three-dimensional signature vector [VA, VB, Is] cleanly separates 13 distinct circuit conditions. A Standardized Nearest-Centroid Classifier achieved 100.00% accuracy on 390 test samples with 5% component tolerances and sensor noise. Both an interactive web app (Streamlit) and a zero-dependency offline desktop GUI (Tkinter) were deployed alongside 22 automated passing unit tests.

## 2. Educational Value for BEEE Students

The project provides a clear, practical demonstration of how first-year electrical engineering laws (Ohm's Law, KCL, KVL, and nodal analysis) integrate directly with computer science concepts (Z-score standardization, Euclidean distance metrics, and streaming online learning). It transforms textbook circuit theory into an engaging, practical engineering diagnostic tool.

## 3. Project Limitations

- DC Steady-State Limitation: The model assumes pure DC steady-state operation and does not analyze AC reactive dynamics (capacitors and inductors).
- Single-Fault Assumption: The system is designed to classify single component failures; concurrent multiple-component faults are not modeled.
- Simulated Sensor Noise: The evaluation relies on mathematical modeling with synthetic Gaussian noise rather than physical ADC quantization.
- Absence of Physical Hardware: While verified against Falstad SPICE simulation, the system was not validated on a physical soldered PCB.

## 4. Future Scope and Enhancements

- **1. AC Reactive Circuits:** AC Reactive Network Extension: Incorporating sinusoidal excitation and measuring impedance magnitude and phase angle (VA ∠ θA) to diagnose capacitor degradation and transformer winding faults.
- **2. Embedded Hardware:** Microcontroller Hardware Deployment: Interfacing an ESP32 microcontroller with a 16-bit ADS1115 ADC to read physical voltages from a breadboard.
- **3. Self-Healing Redundancy:** Automated Self-Healing Circuitry: Utilizing solid-state MOSFET switches to disconnect a diagnosed faulty resistor and switch in a backup redundant component.
- **4. Calibrated Probabilities:** Calibrated Bayesian Probabilities: Implementing Platt scaling to convert Euclidean distances into calibrated posterior failure probabilities.

# References and Team Contributions

## 1. Verified References (IEEE Format)


[1] C. K. Alexander and M. N. Sadiku, Fundamentals of Electric Circuits, 7th ed. New York, NY: McGraw-Hill Education, 2020.

[2] R. L. Boylestad, Introductory Circuit Analysis, 13th ed. Boston, MA: Pearson Education, 2016.

[3] J. W. Bandler and A. E. Salama, "Fault diagnosis of analog circuits," Proceedings of the IEEE, vol. 73, no. 8, pp. 1279–1325, Aug. 1985.

[4] R. Isermann, Fault-Diagnosis Systems: An Introduction from Fault Detection to Fault Tolerance. Berlin: Springer-Verlag, 2006.

[5] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning, 2nd ed. New York, NY: Springer, 2009.

[6] P. Falstad, "Circuit Simulator Applet," Falstad.com, 2023. [Online]. Available: https://www.falstad.com/circuit/ [Accessed: Oct. 2026].

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.


## 2. Team Work Allocation Matrix

| Project Milestone / Task | KOTRA MANOJ KUMAR (23BCE2113) | Mayank Yadav (23BCE0119) |
|---|---|---|
| Electrical Theory & Nodal Analysis | Lead: Hand derivations, KCL/KVL equations, nominal theoretical calculations | Support: Software verification using matrix conductance solver (G·V = I) |
| Falstad Online Circuit Simulation | Lead: Building 13 circuit models in Falstad, capturing digital probes, logging results | Support: Creating automated ASCII schematic exporter and clickable web URLs |
| Machine Learning & Diagnostics | Support: Validating component tolerance bands (5%) and physical constraints | Lead: Implementing Z-score standardization, Nearest Centroid Classifier, and margin score |
| User Interface & Testing | Support: Telemetry checking, UI testing, and conducting manual probe verifications | Lead: Streamlit web app (`app.py`), Tkinter desktop GUI (`gui_tk.py`), and 22 unit tests |
| Documentation & Viva Preparation | Shared: BEEE theory writeup, Viva Q&A preparation, reviewing report calculations | Shared: ML methodology writeup, PPT slides, live demonstration script |

## 3. Essential Run Instructions

**Quick Execution Commands:**

- Install dependencies: pip install -r requirements.txt
- Run 22 unit tests: python -m pytest tests/ -v
- Launch interactive web interface: streamlit run app.py
- Launch offline desktop interface: python gui_tk.py